[Operating Systems]

[(Open Ended Lab) Report]

[Implementation of the Readers-Writers (Part B)]

Submitted by:

[Mudassar Hussain] [23L-6006]

Submitted to:
[Ma’am Sughra Kamran]

[5/5/26]

Department of Electrical Engineering
National University of Computer and Emerging Sciences, Lahore

Table of Contents

Introduction

The ***main idea*** of this project is to implement the **Readers Writer's synchronization problem** using threads and semaphores, and to simulate real-world concurrent access to a shared resource through a dummy database.

In many practical systems such as **hotel reservation systems, airline booking systems, and online databases**, multiple users attempt to access the same data simultaneously. Some users only need to **read data** (e.g., checking room availability), while others need to **modify data** (e.g., booking or cancelling reservations). This creates a synchronization challenge where data consistency must be maintained without unnecessarily restricting access.

The Readers–Writer's problem addresses this challenge by defining rules:

- Multiple readers can access the resource simultaneously.
- Writers must have exclusive access.
- No reading is allowed during writing.

Significance of the Project

This project is significant because:

- It demonstrates how operating systems handle **process synchronization**.
- It models real world systems like **reservation systems and database management systems**.

Understanding and solving such problems is critical for designing **reliable and efficient multi-user systems**.

Problem Analysis

The problem addressed in this project is the **Readers Writer's synchronization problem**, which arises when multiple processes or threads attempt to access a shared resource (database) simultaneously.

In this scenario, there are two types of processes:

- **Readers** → Only read the data
- **Writers** → Modify/update the data

The challenge is to design a system that ensures:

- Multiple readers can access the database at the same time.
- Writers must have exclusive access to the database.
- No reader is allowed to read while a writer is writing.
- No writer can write while readers are accessing the database.

Problem Challenges

Several synchronization issues must be handled:

1. Race Condition

If multiple writers or readers and writers access the database without control, inconsistent or incorrect data may occur.

2. Mutual Exclusion

Writers must be given exclusive access to prevent data corruption.

3. Concurrency Control

Allowing multiple readers simultaneously improves system efficiency, but must be managed carefully.

4. Starvation

In some solutions, writers may wait indefinitely if readers continuously access the database.

Problem Analysis

Solution Strategy

The implementation follows the **Readers-Preference approach**, where:

1. A variable read_count is used to track the number of active readers.
2. A semaphore mutex is used to ensure safe updates to read_count.
3. A semaphore wrt is used to provide exclusive access to writers.

Working Mechanism

Reader Process:

- When a reader enters:
  - It increments read_count
  - If it is the first reader, it blocks writers
- Multiple readers can read simultaneously
- When a reader leaves:
  - It decrements read_count
  - If it is the last reader, it allows writers to proceed

Writer Process:

- A writer must wait until no readers are active
- It locks the resource before writing
- After writing, it releases the resource

Design Requirements

1. Input Requirements

The system takes the following inputs:

- **Number of Reader Threads:** 2
- **Number of Writer Threads:** 1
- **Initial Database Value:** A dummy shared variable (e.g., integer initialized to 0)
- **Thread Execution Behavior:** Continuous execution with simulated delays (using sleep)

These inputs simulate a real-world environment where multiple users (readers and writers) interact with a shared database.

2. Output Requirements

The system produces the following outputs:

- Display messages showing:
  - Which **reader** is accessing the database
  - Which **writer** is updating the database
- Updated value of the shared database after each write operation
- Sequence of execution showing concurrent behavior

3. Functional Requirements

The system must ensure:

- Multiple readers can read simultaneously
- Only one writer can write at a time
- No reader can access the database during writing
- Writers must wait until all reader's finish

4. Constraints

The system operates under the following constraints:

Synchronization Constraints

- Mutual exclusion must be maintained for writers
- Shared variables must be protected from race conditions

Resource Constraints

- Limited number of threads (2 readers, 1 writers)
- Shared memory resource (single database variable)

Logical Constraints

- read_count must always reflect the correct number of active readers
- Semaphores must be used correctly to avoid deadlock

5. Performance Requirements

- Efficient handling of multiple readers
- Minimal delay in accessing the database
- Proper coordination between threads

Feasibility Analysis

1. Time Management Feasibility

The project is feasible in terms of time due to the following reasons:

- The problem is well-defined and based on standard synchronization concepts taught in operating systems.
- Implementation using **threads and semaphores** is straightforward with available libraries such as POSIX threads.
- The project is divided into manageable phases:
    - Understanding the problem
    - Designing synchronization logic
    - Coding and implementation
    - Testing and debugging

2. Cost Management Feasibility

The project is highly feasible in terms of cost:

- No specialized hardware is required.
- Implementation uses **free and open-source tools**, such as:
    - GCC compiler
    - Linux/Unix environment

3. Resource Management

The resources required for the project are minimal:

- **Hardware:** Standard computer system
- **Software:** C/C++ compiler, pthread library
- **Human Resource:** Basic knowledge of operating systems and programming

Possible Solutions

1. Readers-Preference Solution

In this approach:

- Readers are given priority over writers.
- Multiple readers can access the database simultaneously.
- Writers must wait until all reader's finish.

Advantages:

- High performance in read-heavy systems
- Maximum concurrency for readers
- Simple to implement

Disadvantages:

- Writers may suffer from **starvation**
- Continuous arrival of readers can delay writers indefinitely

2. Writers-Preference Solution

In this approach:

- Writers are given priority over readers.
- If a writer is waiting, no new reader is allowed to start reading.

Advantages:

- Prevents writer starvation
- Ensures timely updates to the database

Disadvantages:

- Readers may experience delays
- Reduced concurrency for reader

Possible Solutions

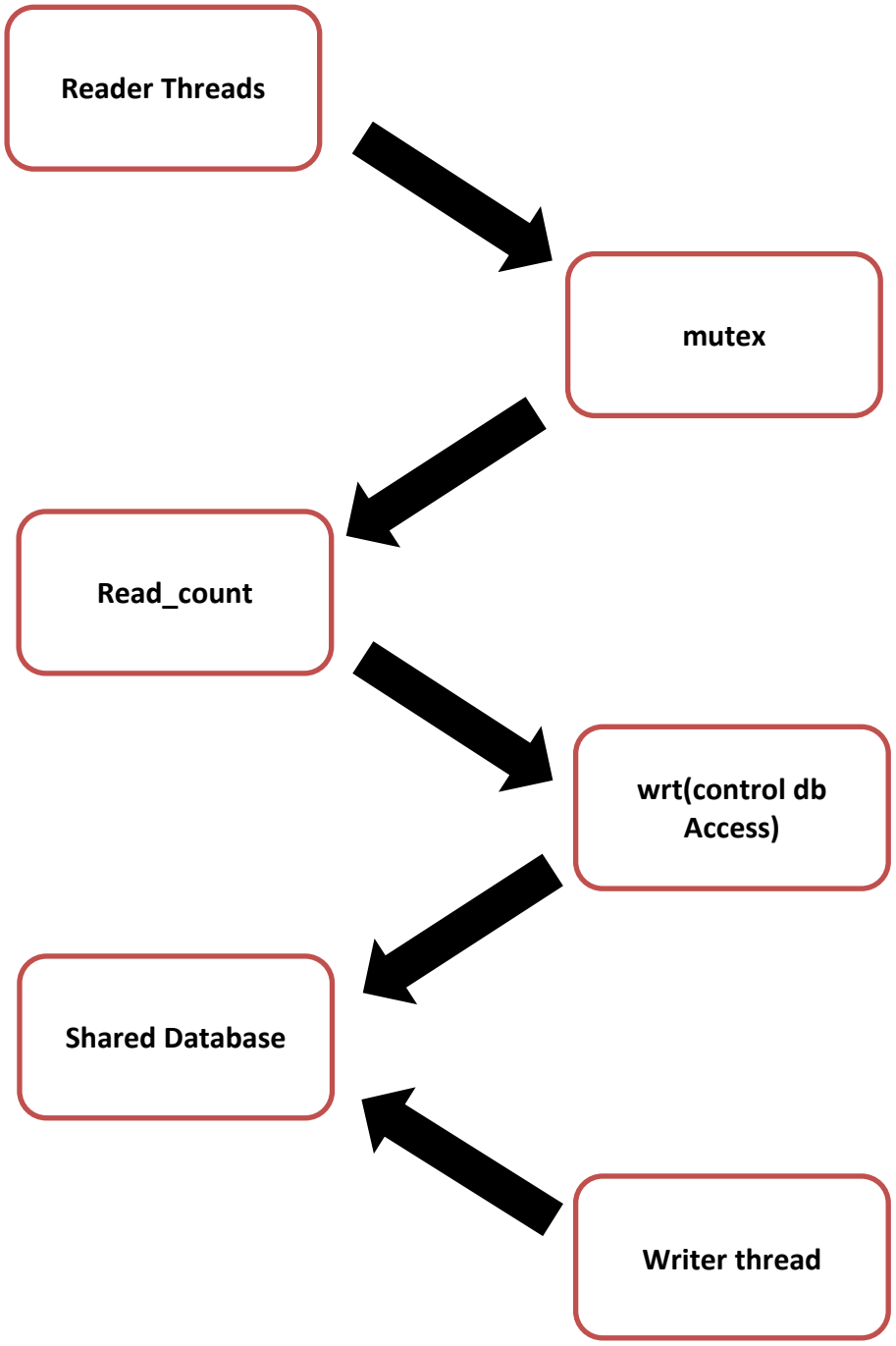
Chosen Solution

For this project, the **Readers-Preference solution** is selected.

Justification:

- The implementation is simple and suitable for learning synchronization concepts.
- It provides high efficiency when the number of readers is greater than writers.
- It meets the basic requirements of the problem:
    - Multiple readers can read simultaneously
    - Writers get exclusive access

Preliminary Design



Design Description

1. Reader Threads Block

This block represents the **five reader processes** in the system.

- Each reader attempts to access the shared database.
- Multiple readers are allowed to read simultaneously.
- Before accessing the database, readers must follow synchronization rules.

Role:

- Perform read operations without modifying data
- Coordinate with other readers using shared variables

2. Writer Threads Block

This block consists of **three writer processes**.

- Writers update or modify the shared database.
- Only one writer can access the database at a time.
- Writers must wait if readers are currently reading.

Role:

- Ensure safe and exclusive modification of shared data

3. Mutex Block

The mutex semaphore is used to protect the shared variable read_count.

- It ensures that only one thread at a time can update read_count.
- Prevents race conditions when multiple readers enter or leave.

Role:

- Maintain consistency of read_count
- Provide mutual exclusion for reader entry/exit

Design Description

4. Read Count Block (read_count)

This block keeps track of how many readers are currently accessing the database.

- Incremented when a reader starts reading
- Decremented when a reader finishes

Working:

- If read_count == 1, the first reader blocks writers
- If read_count == 0, the last reader allows writers

Role:

- Control access between readers and writers

5. Writer Semaphore Block (wrt)

The wrt semaphore ensures exclusive access to the shared database.

- Locked by writers before writing
- Also locked by the first reader to block writers

Role:

- Prevent simultaneous writing
- Prevent reading during writing

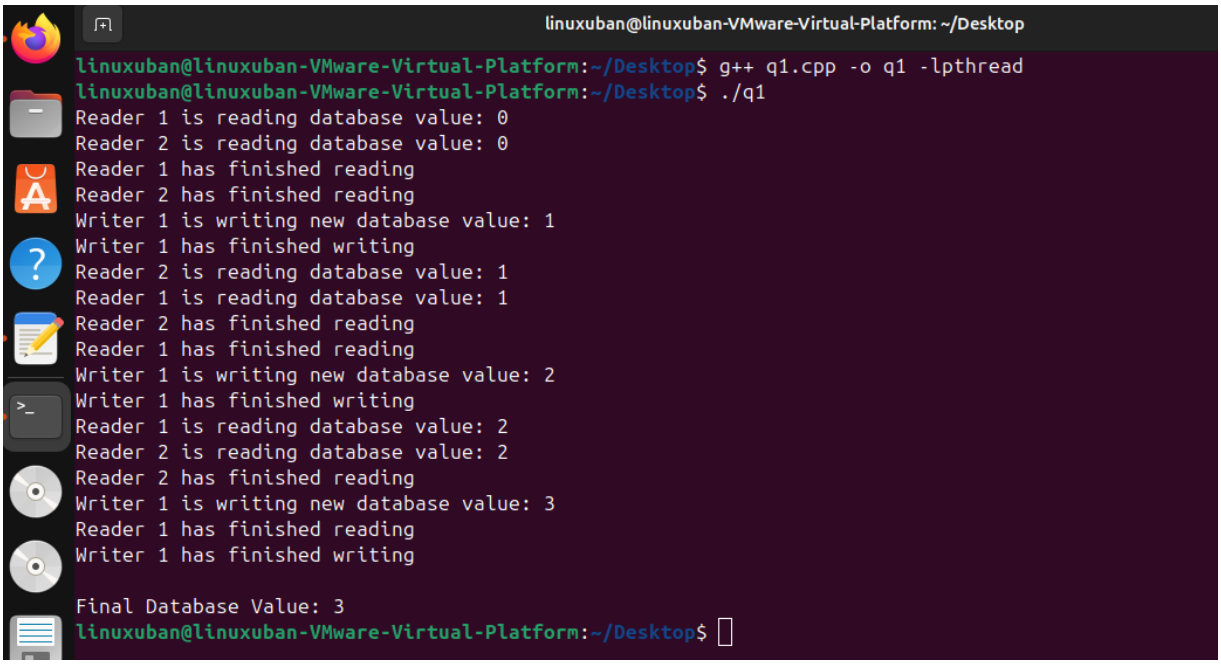
6. Shared Database Block

This represents the common resource accessed by all threads.

Role:

- Simulate real-world shared resources like:
  - Reservation systems
  - Databases
  - File systems

Software Simulation (Optional)



Experimental Results

1. Execution Behavior

During execution, the following behavior was observed:

- Multiple readers were able to read the database **simultaneously**.
- Writers accessed the database **exclusively**, i.e., no other reader or writer was active during writing.
- Readers waited when a writer was writing.
- Writers waited until all readers completed their operations.

2. Observations

- **Concurrent Reading:**

  At several points, more than one reader accessed the database at the same time.
- **Mutual Exclusion for Writers:**

  Only one writer updated the database at a time.
- **Reader–Writer Exclusion:**

  No reader accessed the database while a writer was writing.
- **Correct Data Updates:**

  The database value increased sequentially, indicating correct write operations.

3. Result Verification

The obtained results confirm that:

- The synchronization mechanism is working correctly.
- Semaphores are properly controlling access.
- The system satisfies all constraints of the Readers Writer's problem.

Performance Analysis

The results obtained from the implementation of the Readers Writer's problem demonstrate that the synchronization mechanism using semaphores works correctly in managing concurrent access to a shared database.

1. Correctness of Synchronization

The primary objective of the system is to ensure safe access to the shared database.

- The results confirm that **mutual exclusion for writers is strictly maintained**, as only one writer updates the database at a time.
- Readers are allowed **concurrent access**, and multiple reader threads were observed reading the database simultaneously.
- No scenario was observed where a reader accessed the database during a write operation.

2. Writer Performance

- Writers require exclusive access to the database.
- When readers are active continuously, writers must wait until all readers' finish.
- This results in **delayed write operations** in some execution scenarios.

3. Resource Utilization

- CPU utilization is efficient due to lightweight thread operations.
- Semaphore-based locking avoids busy waiting, making the solution resource-friendly.
- The system uses minimal memory since only a single shared variable (dummy database) is maintained.

4. Scalability Analysis

- The system can be extended to more readers and writers without major changes.
- However, increasing the number of threads may increase waiting time for writers under heavy reader load.

Conclusion

The Readers Writer's problem was successfully implemented using **multithreading and semaphores**, demonstrating an effective synchronization mechanism for controlling concurrent access to a shared resource (dummy database). The developed solution ensures that multiple readers can access the database simultaneously while maintaining exclusive access for writers.

From the results and performance analysis, it is evident that the system maintains **data consistency, correct synchronization, and efficient concurrency for reader operations**. The use of mutex and wrt semaphores effectively prevents race conditions and ensures proper coordination between threads.

In conclusion, the experiment successfully achieves its primary objectives of demonstrating process synchronization and concurrent programming concepts. It provides a clear understanding of how operating systems manage shared resources in multi-process environments. For real-world applications such as database systems or reservation systems, a **fair or writer-priority solution** would be more appropriate to ensure balanced access for all processes.

References

[1] Silberschatz, A., Galvin, P. B., & Gagne, G.
Operating System Concepts (9th / 10th Edition). Wiley.
– Standard textbook covering process synchronization, semaphores, and the Readers–Writer's problem.

[2] Tanenbaum, A. S., & Bos, H.
Modern Operating Systems (4th Edition). Pearson.
– Detailed explanation of concurrency, critical section problems, and synchronization techniques.

[3] POSIX Threads (pthreads) Documentation
– Official reference for multithreading implementation in C.

[4] Google AI

Range of Complex Engineering Activity

<table>
<tr>
<td>EA1: Range of resources
EA2: Level of interaction
EA3: Innovation
EA4: Consequences for society and environment
EA5: Familiarity</td>
<td>

EA1: Range of resources -- The design involves the use of diverse resources, such as, money, equipment, information, and technology.
EA2: Level of Interaction – The design requires resolution of various problems arising from interactions between wide-ranging or conflicting technical or other issues.
EA3: Innovation – The design addresses sustainability and reduces cost requiring creative use of engineering principles and research-based knowledge in novel ways.
EA5: Familiarity – The design activity emphasizes the integration of existing knowledge and tools (or familiar solutions) with new or unfamiliar challenges

<table>
<tr><th>Rubrics</th><th></th><th>LLOs</th><th>Marks</th></tr>
<tr><td>Explains the design process including engagement of resources clearly</td><td>EA1</td><td>LLO3</td><td></td></tr>
<tr><td>Demonstrate the final design clearly with all supporting information</td><td>EA1, EA5</td><td>LLO4</td><td></td></tr>
<tr><td>Demonstrate how innovation has been used in design to resolve conflicting requirements including the impact on society and environment</td><td>EA2, EA3</td><td>LLO4</td><td></td></tr>
<tr><td>Prepares a report which explain the design process including engagement of resources clearly, and is free from grammatical errors.</td><td>EA1</td><td>LLO3</td><td></td></tr>
</table>
</td>
</tr>
</table>